BookCast

Software Requirements Specification (SRS)

Version: 1.0
Date: December 10, 2025

Abstract

This document defines the functional and non-functional requirements of the BookCast MVP. BookCast is a mobile-first platform that transforms book content into AI-generated podcast episodes featuring customizable virtual hosts.

Contents

1 Introduction

BookCast is a mobile-first application that converts user-uploaded books into AI-generated podcast episodes. These episodes are narrated by customizable "Virtual Podcasters" capable of monologues, dual-host, or group conversations.

2 Purpose

The purpose of this SRS is to outline requirements defining the BookCast MVP. It includes system behavior, constraints, workflows, and architectural expectations.

3 System Scope and Overview

The BookCast system uses a mobile-client and cloud-backend architecture:

- **Frontend:** React Native (iOS/Android).
- **Backend:** Node.js (NestJS) API gateway.
- **AI Processing:** Python microservices for LLM script generation and TTS synthesis.
- **Storage:** PostgreSQL + Cloud Object Storage (AWS S3/GCP Storage).
- **Async Jobs:** RabbitMQ to queue heavy generation tasks.

4 Functional Requirements

4.1 User Accounts & Authentication

FR-1 System shall support login via email/password or Google/Apple OAuth.

FR-2 System shall provide optional 2FA (SMS or authenticator app).

FR-3 Users shall have a profile showing settings and created episodes.

4.2 Virtual Podcaster Management

FR-4 Users shall create one or more AI virtual podcasters.

FR-5 Each podcaster shall have configurable name, avatar, personality, and speaking style.

FR-6 System shall offer predefined personality templates.

FR-7 Users shall assign one or more TTS voices with accents, tone, and style.

FR-8 Podcaster speech settings shall support parameters such as: pace, pitch, emphasis.

4.3 Book Ingestion

FR-9 Users shall upload PDF/EPUB files.

FR-10 Users shall input URL sources for extraction.

FR-11 System shall extract clean text from uploaded materials.

FR-12 (Optional) Users may import files from Google Drive or Dropbox.

4.4 Chapter & Content Selection

FR-13 Users shall select full book, chapters, or page ranges.

FR-14 Users shall preview extracted text before processing.

FR-15 Users shall combine multiple sections into one episode.

4.5 Episode Generation Workflow

FR-16 Users shall select episode type (Monologue, Dual-host, Group).

FR-17 LLM shall generate scripts based on text + podcaster personalities.

FR-18 TTS shall convert scripts into audio.

FR-19 System shall store script and audio as final output.

4.6 Episode Feed & Social Features

FR-20 Episodes shall appear in a scrollable feed.

FR-21 Users shall play audio in-app.

FR-22 Users shall like, comment, and bookmark episodes.

FR-23 Users may follow podcasters.

4.7 Search & Discovery

FR-24 Search shall support: title, author, keywords, podcaster name.

FR-25 Discover section shall show trending episodes.

4.8 Content Management

FR-26 Users shall view a history of uploaded books and episodes.

FR-27 Users shall edit and re-generate episodes.

FR-28 Users may download audio files.

5 Non-Functional Requirements

5.1 Performance

NFR-1 App UI shall run at 60fps where possible.

NFR-2 Episode generation shall complete asynchronously and notify user.

NFR-3 API latency should not exceed 500ms average.

5.2 Scalability

NFR-4 Backend shall support horizontal scaling (Docker/Kubernetes).

NFR-5 RabbitMQ must decouple heavy AI tasks.

NFR-6 Database must scale vertically and horizontally.

5.3 Availability & Reliability

NFR-7 Target uptime: 99.9%.

NFR-8 Deployments shall use multi-zone redundancy.

NFR-9 System shall support automated backups.

5.4 Security

NFR-10 TLS encryption required for all transmitted data.

NFR-11 Sensitive data stored encrypted (AES-256, bcrypt).

NFR-12 Input validation required across all APIs.

NFR-13 System shall comply with GDPR/CCPA data rights.

5.5 Usability & Maintainability

NFR-14 User flow must remain simple: Upload $\rightarrow$ Configure $\rightarrow$ Generate.

NFR-15 Codebase must maintain modular structure with tests.

6 Technical Architecture

6.1 Mobile Frontend

- React Native (TypeScript)
- Handles UI, file upload, episode playback, API calls

6.2 Backend API Gateway

- NestJS (Node.js)
- Handles routing, authentication, business logic, RabbitMQ task dispatch

6.3 Data Layer

- PostgreSQL for relational data
- Cloud Object Storage for audio/books
- Redis (optional) for caching

6.4 AI Microservices (Workers)

- Python services consuming RabbitMQ jobs
- LLM script generator
- TTS generator (Polly, Google TTS, or similar)

7 Backend Services Overview

The backend consists of the following logical services:

- **Auth Service** – Manages registration, login, tokens, and 2FA.
- **User Service** – Profiles, settings, preferences.
- **Podcaster Service** – Creation and configuration of virtual hosts.
- **Book Ingestion Service** – Handles PDF/EPUB upload, extraction, cleaning.
- **Episode Service** – Script generation request routing, retrieval, updates.
- **AI Worker Service** – Python workers generating scripts and TTS audio.
- **Feed/Discovery Service** – Sorting, trending logic, recommendations.
- **Social Interaction Service** – Likes, comments, bookmarks.

8 API Specifications (High-Level)

API-1 `POST /auth/login`, `/signup`

API-2 `POST /books`, `GET /books/:id`

API-3 `POST /podcasters`

API-4 `POST /episodes`, `GET /episodes/:id`

API-5 `GET /feed`

API-6 `POST /episodes/:id/comments`

9 Security Requirements

SR-1 OAuth flows must use PKCE for mobile clients.

SR-2 System must use short-lived access tokens + refresh tokens.

SR-3 Critical events (login, regeneration, password changes) shall be logged.

SR-4 API endpoints shall have rate limits.

10 Deployment & Infrastructure

- Docker containers for all services
- Deployment on AWS/GCP (EKS, GKE or Cloud Run)
- Managed database (RDS/Cloud SQL)
- CDN for serving audio at scale
- CI/CD using GitHub Actions or GitLab CI

11 Future Enhancements

- Multi-language support
- Voice cloning
- RSS export for Spotify/Apple Podcasts
- Premium subscription model

12 Glossary

LLM Large Language Model used for script generation.

TTS Text-to-Speech engine producing audio.

RabbitMQ Message broker for async workloads.

NestJS Backend framework used for API architecture.

PKCE Security enhancement for OAuth flows.